

Broadway Property Management

Platform Status & Roadmap

Built · Gaps · Path to revenue

Table of Contents

1. Executive summary
2. What has been built (evidence in repo)
3. Deployment status (your current work)
4. What must be corrected (before charging money)
5. What is missing — backend
6. What is missing — frontend
7. Are bank and MoMo APIs “ready”?
8. Intelligent feature: payment proof vs bank statement
9. Reminders: email and WhatsApp
10. Mobile app store readiness
11. Roadmap to “top-notch” and revenue (recommended phases)
12. Is the platform “feature enough” to start making money?
13. Immediate next actions (this week)
14. Document index

Executive summary

Broadway Property Management is a Rwanda-focused property SaaS (landlords, tenants, accountants, platform operators) with a working web MVP. The product is suitable for pilot demos and early paying landlords once hosting, security, and billing collection are finished.

Area	Current status
GitHub repository	Live — monorepo (`backend/` + `frontend/`)
Frontend (web)	Deployed on Vercel (Next.js 16)
Backend (API)	In progress — DigitalOcean App Platform (not yet production-stable)
Database	Dev DB on DO planned; production should use managed Postgres with backups
Mobile apps (iOS/Android)	Not started — web only
Bank / MoMo live APIs	Not integrated — manual proof workflow only
Intelligent payment reconciliation (OCR)	Not built — recommended Phase 2
SaaS subscription charging	Plan catalog only — no card/MoMo billing for Broadway fees

Verdict: Strong demo / pilot platform. Not yet top-tier revenue-ready hardening and partner API agreements.

What has been built (evidence in repo)

2.1 Core product (working in code)

- Multi-tenant accounts: platform operator + landlord workspaces
 - Role-based access: PLATFORM_OWNER, OWNER, ACCOUNTANT, LAWYER, TENANT
 - Properties (buildings, UPI), units, tenant profiles, lease PDFs (upload, versions, approve)
 - Rent invoices, manual bank / MoMo payment proof upload, owner review (approve/reject)
 - Rwanda tax profile + obligation tracker + PDF export (not live RRA filing)
 - Expenses with receipt upload, annual forecast, analytics dashboards
 - Tenant portal: view leases, pay rent (proof flow)
 - Platform operator: create/suspend landlord clients, rolled-up KPIs
 - Audit log API, marketing site, pricing from plan catalog
 - Google + Microsoft sign-in (when env vars set)
 - JWT access + refresh tokens, team invites

2.2 Infrastructure & docs

- GitHub repo with README, SETUP, DEPLOY_STAGING, PARTNER_DEMO, DEVELOPER_HANDOFF

- render.yaml blueprint (alternative to DigitalOcean for API)
- Demo seed script: backend/scripts/seed-demo-staging.js
- PDF generator for developer handoff

2.3 Subscription plans (catalog only — not charged automatically)

Plan	Indicative price (RWF/mo)	Target
Starter	35,000	Solo landlord, ~25 units
Professional	95,000	Growing portfolio + accountant
Business	185,000	Property management firm
Enterprise	Custom	Banks, REITs, large operators
Platform Partner	Per workspace	B2B2B resellers

Plans are shown at signup; payment collection fo

Deployment status
(your current work)

Component	Status	Action needed
GitHub	Done	Keep `main` protected; use PRs for changes
Frontend — Vercel	Done (per your update)	Set `NEXT_PUBLIC_API_BASE_URL` to **production API URL** after backend is live
Backend — DigitalOcean	In configuration	Push env vars (`\${broadway_pm.*}`), JWT secrets `CORS_ORIGINS` build/run
Database	DO dev DB attached	For revenue: upgrade to managed Postgres, `DB_SYNCHRONIZE=false`, migrate to S3/Spaces before scale (nroofs, leases, receipts)
Uploads	Local disk on API server	
SSL / domains	Pending	Custom domain + HTTPS for API and app
Monitoring	Missing	Sentry, uptime checks, log aggregation

3.1 Backend hosting checklist (DigitalOcean)

- [x] Source directory: backend
- [x] Build: npm install && npm run build
- [x] Run: npm run start:prod
- [x] DB linked via \${broadway_pm.HOSTNAME} etc.
- [] Remove duplicate app-level env (localhost, development, hardcoded passwords)
- [] Strong JWT_SECRET / JWT_REFRESH_SECRET (Encrypt on DO)
- [] CORS_ORIGINS = exact Vercel URL(s)
- [] APP_URL = backend public URL
- [] Seed production DB after first deploy

3.2 Frontend hosting checklist (Vercel)

- [] NEXT_PUBLIC_API_BASE_URL = DigitalOcean (or Render) API URL
- [] NEXT_PUBLIC_GOOGLE_CLIENT_ID matches backend
- [] Add missing public/broadway-logo.png (referenced in UI/manifest)
- [] Token refresh wired (POST /auth/refresh)
- [] Production OAuth UX (not manual token paste)

What must be corrected (before charging money)

Priority	Issue	Risk
P0	JWT secrets still example values on staging	Account takeover
P0	`DB_SYNCHRONIZE=true` in production	Data loss / schema drift
P0	Uploads on local container disk	Lost files on redeploy
P0	No SaaS billing (Stripe, MoMo merchant, or invoice for Broadway fee)	Cannot collect subscription revenue automatically
P0	Payment gateway = `AGGREGATOR_PAYMENT_METHOD` only	Tenants cannot pay via live checkout
P1	CORS wide open if `CORS_ORIGINS` unset	Security
P1	Public payment webhook with no signature verification	Fraud
P1	Rent reminders cron only runs for `DEFAULT_ACCOUNT_ID`	Wrong tenants not reminded
P1	No automated test suite for payments/auth	Regressions
P2	Password reset scans all users	Performance at scale

What is missing — backend

Capability	Built?	Notes
Live bank gateway (BK, BPR, Equity, I&M, etc.)	No	Static bank list + stub `PaymentGatewayService`
MTN MoMo API	No	Toggle + manual proof only
Airtel Money API	No	Same
Payment reconciliation (statement vs proof)	No	Endpoint returns pending list only
OCR / AI document matching	No	No vision/OCR libraries
RRA e-filing API	No	Tracker + PDF export + manual purchase codes
SMS reminders	No	Not in codebase
Email reminders	Partial	Works if SMTP configured
WhatsApp reminders	Partial	Generic webhook URL only — not Meta RSP
S3/object storage for uploads	No	`./uploads/*` on server
DB migrations	No	TypeORM synchronize in dev
Webhook HMAC for PSP	No	Stub accepts any POST
Multi-account reminder scheduler	No	Single `DEFAULT_ACCOUNT_ID`
Broadway subscription charging	No	Plans displayed only

What is missing — frontend

Capability	Built?	Notes
Native iOS / Android app	No	Next.js web only
App Store / Play Store package	No	No Capacitor/React Native
Service worker / offline PWA	No	`manifest.json` only
Real-time sync (WebSocket/SSE)	No	Fetch on page load only
Token refresh in UI	No	Re-login when access token expires
Server-side auth middleware	No	`localStorage` only
Audit log UI	No	API exists
Live MoMo checkout UI	No	Manual proof screens only
i18n (Kinyarwanda / French)	No	English UI
Error monitoring (Sentry)	No	—
E2E tests	No	—
Logo asset in `public`	Missing file	Broken branding in places

6.1 Web flow synchronization

- Web !” API: Same REST API; data refreshes when user opens a page or clicks actions — not real-time.
- Web !” mobile app: N/A — no mobile app. A future app would use the same API; you must add refresh/push separately.
- Timeliness: Rent reminders depend on backend cron (POST / payments/rent-reminders/run or scheduler) + SMTP/WhatsApp config — not guaranteed “on time” without job queue and monitoring.

Are bank and MoMo APIs “ready”?

No — not for live money movement or automation
What exists today:

- Landlord toggles for bank transfer proof, MTN, Airtel (settings)
- Tenant uploads screenshot / reference !' owner manually approves
- POST /payments/gateway-intent returns placeholder AGGREGATOR_PLACEHOLDER

- Static list of Rwanda commercial banks (names only)

What does not exist:

- API credentials, sandbox, or production keys for any bank or telco
- Settlement webhooks that update payment status
- Statement import or auto-matching

Partner outreach: Use the companion document docs/BANK_AND_MOMO_API_PARTNER_REQUEST.md (PDF available) when approaching banks or MTN/Airtel.

Intelligent feature: payment proof vs bank statement

Recommendation: Phase 2 (high value, not in MVP).

Proposed capability:

1. Tenant uploads MoMo/bank transfer proof (image/PDF) — already supported.
2. Owner uploads monthly bank statement (CSV/PDF).
3. AI/OCR extracts date, amount, reference, counterparty.
4. System suggests matches to open invoices; owner confirms.

Why not in MVP: Requires OCR pipeline, Rwanda-specific statement formats, dispute workflow, and data privacy review.

Rough effort: 6–10 weeks (MVP matcher) after core platform is stable.

Tech options: AWS Textract, Google Document AI, or specialized fintech OCR; store artifacts in object storage; human-in-the-loop approval.

Reminders: email and WhatsApp

Channel	Ready for production?	Requirement
Email	Partial	Configure `SMTP_HOST`, `SMTP_USER`, `SMTP_PASS` on API
WhatsApp	Partial	`WHATSAPP_WEBHOOK_URL` must point to **your** BSP (e.g. Twilio, Meta Cloud API Partner)
SMS	No	Cloud API Partner — not built-in
In-app push	No	No mobile app

Rent reminder flow: PaymentsService.notifyTenant
Scheduler must run for all landlord accounts (fix

Mobile app store readiness

Question	Answer
iOS App Store ready?	**No**
Google Play ready?	**No**
PWA installable?	Partial — manifest only, no service worker
Path to stores	Phase 3: React Native or Capacitor wrapper + same API + push notifications + store compliance (privacy policy, data safety, screenshots) Estimated timeline to stores: 3–5 months after web production is stable

Roadmap to “top-notch” and revenue (recommended

phases)

Phase A — Go live (4–6 weeks)

- Finish DO backend + Vercel !" API connection
- Production Postgres, migrations, DB_SYNCHRONIZE=false
- Object storage for uploads
- Security: JWT, CORS, encrypted env, rate limits
- SMTP for email; WhatsApp BSP contract
- Manual payments OK for first 10–20 landlords
- Invoice landlords monthly for Broadway subscription (manual bank transfer) until automated billing

Phase B — Collect Broadway revenue (4 weeks)

- Stripe or Flutterwave/Paystack for SaaS subscription (RWF/international)
- Or MoMo merchant API for monthly plan payment
- Enforce plan limits (units/seats) in API
- Legal: terms, privacy, DP law, refund policy

Phase C — Payment automation (8–12 weeks)

- Sign 1 bank or payment aggregator (see partner PDF)
- MTN MoMo Collections API (tenant rent)
- Webhook signature verification + reconciliation v1

Phase D — Intelligence (6–10 weeks)

- Statement upload + OCR matching (Section 8)

Phase E — Mobile (12+ weeks)

- React Native app, push reminders, store release

Is the platform “feature enough” to start making money?

Yes — for controlled pilot revenue with manual payment proof and

- Target: 5–20 landlord workspaces in Kigali
- Price: Starter / Professional plans (manual monthly invoice)
- Support: WhatsApp group + email

Not yet for:

- Fully automated rent collection via MoMo/bank APIs
- Hands-off reconciliation
- App Store presence
- Enterprise bank-grade SLAs

Competitive edge today: Rwanda tax tracker, UPI fields, multi-role v operator model, bilingual roadmap.

Competitive gap: Live payments, AI reconciliation, mobile app, auto

Immediate next actions (this week)

- 1. Complete DigitalOcean deploy; point Vercel API URL to DO backend.
- 2. Run seed-demo-staging.js on production DB; run partner demo.
- 3. Replace JWT secrets; fix CORS to Vercel domain only.
- 4. Configure SMTP; test password reset and rent reminder email.
- 5. Sign first 1–3 pilot landlords (manual onboarding).
- 6. Share BANK_AND_MOMO_API_PARTNER_REQUEST.pdf with BK/BPR/MTN when ready for Phase C.

Document index

Document	Purpose
`PLATFORM_STATUS_AND_ROADMAP.md` / `.pdf`	This file
`BANK_AND_MOMO_API_PARTNER_REQUEST.md` / `.pdf`	Institution outreach
`DEVELOPER_HANDOFF.md`	Technical handoff
`DEPLOY_STAGING.md`	Vercel + Render/DO deploy
`SETUP.md`	Local development

*Generated for Broadway Property Management — internal & partner use only